

Abbreviations

- HDD - Hard Disk Drive
- SSD - Solid State Drive
- USB - Universal Serial Bus
- CD - Compact Disk
- DVD - Digital Versatile Disc
- ATA - AT(Advanced Technology) Attachment
- PATA / IDE - Parallel AT Attachment / Integrated Device Electronics
- SATA - Serial AT Attachment
- SCSI - Small Computer System Interface
- SAS - Serial Attached SCSI
- PCIe - Peripheral Component Interface - Express
- NVMe - Non-Volatile Memory Express
- KVM - Kernel-based Virtual
- MBR - Master Boot Record
- GPT - GUID Partition Table
- GUID - Globally Unique Identifier
- BIOS - Basic Input Output System
- UEFI - Unified Extensible Firmware Interface
- GrUB - Grand Unified Bootloader
- XFS - eXtended File System
- FHS - Filesystem Hierarchy Standard
- LVM - Logical Volume Manager
- PV - Physical Volume
- VG - Volume Group
- LV - Logical Volume

Bootling

- Process from computer power on to OS startup.

Bootable device

- Storage device (disk, pen drive, CD/DVD, etc.) whose boot block contains bootstrap program, is said to be Bootable device.

Bootstrap program

- Bootstrap program can load OS kernel into RAM and start its execution.
- Bootstrap program is different for each OS each version.
- Bootstrap is located in first sector (512 bytes) of a disk/partition.

Bootloader program

- Bootloader program can be configured to show multiple boot options to end user.
- Depending on user selection, it runs Bootstrap program of corresponding OS.
- There are many important bootloader programs.

- ntldr (Before Windows Vista) --> boot.ini
- "bootmgr" (Windows Vista Onwards) --> bcd (boot config data)
 - (admin command prompt) cmd> bcdedit
- LiLo --> Linux Loader
- "GrUB" (Grand Unified Boot Loader) --> menu.lst or grub.cfg
- BTX (BooT eXtended) --> BSD Unix
- SILO (Sparc Interactive Loader) --> Solaris
- Bootcamp --> Mac OS X
- "uBoot" --> Linux bootloader for Embedded
- The Bootloader has some config file associated with it.

Bootstrap loader

- The set of programs fixed in Base ROM (Motherboard) is called as "Firmware".
 - BIOS (Basic Input Output System) -- Firmware developed for PC by IBM + MS.
 - EFI (Extensible Firmware Interface) -- Firmware developed for PC by Intel + HP.
- Bootstrap loader is a program from Base ROM.
- It find the bootable device as per "boot device priority" in the BIOS setup.
- Once bootable device is found, it starts its bootloader.

Booting steps

- Computer power on.
- Load firmware (BIOS or EFI) programs from base ROM into RAM.
- Run POST/BIST (from firmware).
- Run Bootstrap loader (from firmware) to find bootable device.
- Bootstrap loader loads bootloader program from bootable device.
- Bootloader program shows multiple options to end user and user select one of them.
- Bootloader program run corresponding bootstrap program.
- Bootstrap program load OS kernel into main memory and OS boot
- Startup Process
 - Linux kernel access Root file system.
 - systemd/init (sbin directory) process is executed (pid=1).
 - systemd is designed to start (independent) services parallely. In init services were started sequentially.
 - It starts multiple child processes as per configred run-level.
 - Runlevels are like "checkpoints"/"states".
- Runlevels
 - 1 - Single user mode (Used for failsafe/rescue)
 - 2 - Multi user mode (User login enabled, User files accessible)

- 3 - Networking (Multiuser with Networking on CLI) -- multi-user.target
- 4 - Reserved (unused)
- 5 - Graphical user interface -- graphical.target
- <https://likegeeks.com/linux-runlevels/>
- Linux commands
 - terminal> runlevel
 - terminal> init 0 --> shutdown
 - terminal> init 6 --> reboot
 - terminal> init 3 --> go to runlevel 3 (GUI will be stopped if already started)
 - terminal> init 5 --> go to runlevel 5 (GUI will be started)
 - terminal> startx --> start GUI (give this command from runlevel 3)

BIOS = Basic Input Output System

- It is a standard for firmware interfaces stored on a computer motherboard's ROM chip
- The BIOS firmware runs when the computer powers on, enabling it to test the various hardware components in a computer and run the boot loader to start the operating system
- The BIOS has access to the ports used by basic hardware input devices like a mouse and keyboard. Users can also load up a BIOS interface instead of an operating system to make various hardware-level changes
- BIOS was the dominant standard for home and enterprise computers for several decades
- Functions of BIOS
 - Initialize hardware (CPU, RAM, disk)
 - Perform POST (Power-On Self Test)
 - Load bootloader from disk

UEFI = Unified Extensible Firmware Interface

- Modern replacement for BIOS
- It is newer firmware technology that has largely replaced BIOS by bringing with it several key advantages
- UEFI runs faster than BIOS, can operate within a greater amount of memory, can access storage drives of currently unattainable sizes, can access more hardware types, and has improved security protections
- Most modern motherboards, as well as the pre-assembled PCs that use them, ship with UEFI. The secure boot features of UEFI are critical
- Like BIOS, UEFI provides an environment to execute a boot loader and ultimately start up the operating system
- Functions of UEFI
 - Initialize hardware
 - Load OS
 - Provide advanced features
- EFI Partition
 - Special partition:/boot/efi
 - Stores bootloader files
 - Used by UEFI
- Features

- Secure Boot
- Faster startup
- Better hardware support
- Network boot capability

GrUB

- The boot loader uses three main components that work together to load the operating system in stages:
 - The boot sector program is loaded by a boot environment on startup and has a fixed size of 512 bytes. Its main function is to load the second-stage boot loader; however, it can also load another sector or a kernel
 - The second-stage boot loader loads the operating system and contains a kernel loader
 - Finally, the boot loader installer controls the installation of drive sectors and can only be run when booting from a drive. It coordinates the activities of the boot sector and the boot loader

SysV init – Boot Process (Step-by-Step)

- Step 1: System Power ON
 - BIOS/UEFI runs
 - Hardware is initialized
- Step 2: Bootloader Executes
 - Bootloader (like GRUB) loads Linux kernel into memory
- Step 3: Kernel Initialization
 - Kernel initializes:
 - CPU
 - Memory
 - Devices
 - After initialization:
 - Kernel starts init process
- Step 4: Init Process Starts (PID = 1)
 - Kernel executes:
 - /sbin/init
- Step 5: Read Configuration File
 - /etc/inittab
 - Init reads:
 - Default runlevel
 - What to execute
- Step 6: Determine Default Runlevel
 - Example:
 - id:3:initdefault:
 - System enters:
 - Runlevel 3 (multi-user CLI)
- Step 7: Execute Startup Scripts
 - Based on runlevel, init executes scripts from:
 - /etc/rc.d/rcX.d/
 - (X = runlevel)

- Step 8: Script Execution Order
 - Scripts are executed sequentially
 - Naming:
 - S10network
 - S20ssh
 - K10httpd
- Step 9: Start Services
 - Examples:
 - Network
 - SSH
 - Database
 - One by one (no parallel execution)
- Step 10: System Ready
 - After all scripts executed:
 - Login prompt appears
 - System ready for user

systemd – Boot Process (Step-by-Step)

- Step 1: System Power ON
 - BIOS/UEFI starts
 - Hardware initialization
- Step 2: Bootloader Executes
 - Bootloader (GRUB) loads Linux kernel into memory
- Step 3: Kernel Initialization
 - Kernel initializes:
 - CPU
 - Memory
 - Devices
 - After initialization
 - Kernel starts systemd
- Step 4: systemd Starts (PID = 1)
 - Kernel executes:
 - /usr/lib/systemd/systemd
 - This is the first user-space process
- Step 5: Load Default Target
 - systemd reads default target from:
 - /etc/systemd/system/default.target
- Step 6: Identify Target
 - Example:
 - multi-user.target
 - Equivalent to:
 - Runlevel 3 (CLI mode)
- Step 7: Load Unit Files
 - systemd loads unit files from:
 - /etc/systemd/system/
 - /lib/systemd/system/

- Step 8: Resolve Dependencies
 - systemd checks:
 - Which services depend on others
 - Which can run in parallel
- Step 9: Parallel Service Startup
 - Unlike SysV:
 - systemd starts services in parallel
 - Faster boot
 - Efficient resource usage
- Step 10: Activate Required Units
 - Types:
 - .service → services
 - .mount → filesystems
 - .socket → communication
- Step 11: Mount Filesystems
 - systemd mounts required filesystems automatically
 - Uses configuration like:
 - /etc/fstab
- Step 12: Start System Services
 - Examples:
 - Network
 - SSH
 - Logging
- Step 13: System Ready
 - Final state:
 - Target reached
 - Login prompt / GUI appears